```
unsigned long startTime = 0;
unsigned long interval = 500;
```

This declares our counter and interval numbers in the unsigned long data type to avoid any weirdness with performing arithmetic or comparison operations using the `millis()` function.

```
if (startTime + interval < millis()) {
```

This line starts our counter by adding our interval to the start time and checking to see if this value has been exceeded by the time recorded by the `millis()` function. If `millis()` is the larger number, then the allotted time has passed and we should execute the enclosed statements. If the interval hasn't passed then we will skip this block of code and continue on to the rest of the sketch, which for this example is empty.

```
state = !state;
digitalWrite(ledPin, state);
startTime = millis();
```

Inside the `if` statement, these three lines begin by toggling the state of the LED so that if it starts as `LOW` it will now be `HIGH` and vice versa. The second line outputs whatever state we are on currently to the LED pin. Then we end the block of code by starting our timer over again, assigning the current time to our new start time.

By having multiple intervals and multiple start times we could theoretically have multiple or even staggered "delays" happening simultaneously with no real impact to any of them. As neat as this is, this technique is not necessarily a better `delay()` because there will be times when disabling inputs or otherwise stopping the normal program flow is necessary, or if the added complexity is simply not worth it. Debouncing a switch in code, as shown earlier, is one good example where it is necessary to stop reading the inputs for a very brief time just to make sure the signal received was the intended signal.

micros()

Where the `millis()` function returns the current operating time in milliseconds, the `micros()` function does the same, but in microseconds. This could be used in exactly the same manner as `millis()`, just on a much smaller scale, effectively returning the value 1000 for every 1 that `millis()` would return.

Unlike `millis()`, `micros()` will overflow, or reset back to 0, every 70 minutes or so. It also only has a resolution of 4 microseconds on the Arduino Uno, meaning that every value returned by the `micros()` function will be a multiple of 4.

Random Functions

Armed with a decent grasp of how to handle the Arduino delay functions, as well as an advanced method for creating a delay without one, let's look now at some ways to make things less precise with a little randomness. Up to this point, our timing has been fairly straightforward: turn on the LED, wait for 1 second, turn off the LED, wait for another second, and so forth. Each of our delays has been specified to a relatively accurate degree of precision so that a delay of 1000 milliseconds will be reasonably sure to give us a delay of 1 second. But what if we wanted a delay that was somewhere between 250 and 1000 milliseconds and not only that, but this value changed somewhat randomly every time it is accessed?